

Domain, Department Domain & Search-Term Selection

Enrichment pipeline — logic & rules reference · `enrichment_api`

All three fields are computed during **finalisation** in `orchestrator.py`, with helpers in `website_resolver.py` and `search_terms.py`. The public **"Domain"** output column is actually `website_url`; the bare registrable `domain` is internal — it is used to derive search terms and to gate the department probe.

1 · Institution `domain` and `website_url`

Two related-but-distinct fields, both starting `None`. Resolution follows a strict precedence — the first non-empty source wins, because `website_url` is only written while empty.

Resolution order (first non-empty wins)

Path A — ROR (highest precedence, set during Tier 1)

ROR carries no domain field; it is derived from ROR's website link:

- `extract_website_from_ror` → first `links[]` entry with `type == "website"`.
- `extract_domain(website)` → registrable domain from `urlparse().hostname`, last two labels, with a hardcoded table of two-part TLDs (`co.uk`, `ac.uk`, `ac.jp`, `com.au`, ...) that take three.
- Writes `website_url` (raw ROR URL) and `domain`. The person-affiliation path does the same after ROR-confirming the web-proposed institution.
- **Companies via GLEIF/LEI**: LEI has no website, so ROR's domain/website survive — LEI can overwrite `name1` but never the domain.

Paths B & C (`_maybe_resolve_website_bc`) run **only if** `website_url` is still empty — so ROR always wins — and are skipped when `name1` is blank.

Path B — SERP resolution (`resolve_website_via_serp`, any record type)

- Query: `"{name1}" official website` + geo — research institutions add country; companies/unknown add `city state` (else country, else nothing). `num_results=5`.
- Candidates filtered to *valid*: URL matches `^https?://`, **not blacklisted**, and passes **name-overlap** (a ≥4-char significant token of `name1` appears in URL or title).
- **Research institution**: top-ranked valid result. Confidence `high` only if TLD ∈ `{edu, gov, org}`, else `low`. URL reduced to `scheme://host`.
- **Company/unknown**: rank each — `0` = name only in title (not host), `1` = name in host but a foreign brand word introduced, `2` = clean name-in-host. Max wins. **Rank 0 → rejected** (defers to Path C). Confidence `high` at rank 2, `low` at rank 1.
- Write: sets `website_url`; if confidence ≠ high, flags *"resolved by SERP with low confidence — verify."*

Path C — LLM inference (`infer_website_via_llm`, only if B found nothing)

- One LLM call with `name1 + city/state/country`. Treats `"/null/none/unknown/n-a` as no result; validates URL shape.
- Always `low` confidence → writes `website_url` and **always flags** *"inferred by LLM — verify."*

Domain back-fills (only when `domain` is still empty)

- From `website_url` right after B/C: `extract_domain(website_url)` — how ROR-less rows get a domain.
- From `source_url` in `finalise` (a successful Tier 2A/2B faculty/contact page) — last resort.
- A website-derived domain **never overwrites** a ROR domain (every back-fill is guarded by `not result.get("domain")`).

Guards against a wrong domain

- **Blacklist** (Path B only): wikipedia, linkedin, facebook, twitter/x, instagram, youtube, ratemyprofessors, glassdoor, yelp, bbb, crunchbase, bloomberg, indeed, ziprecruiter — host or any subdomain.
- **Name-overlap** required to be valid; **name-in-host** required for companies (else rank 0 → rejected) — stops emitting a stranger's domain.
- **Foreign-brand guard**: for hyphenated hosts, a ≥4-char label part that isn't a prefix of a `name1` token downgrades it (`siemens.com` beats `siemens-healthineers.com` for "Siemens AG").
- **TLD authority**: only `edu/gov/org` grant `high` confidence for institutions.
- **Country guard on ROR** upstream — a wrong-country ROR org is rejected before its domain is ever read.

2 · department_domain (the dept subdomain / URL probe)

Resolved entirely by `_probe_department_url`, run on every return path. It writes `department_domain` directly and never touches `website_url`. (`derive_department_domain` in `search_terms.py` is dead code — only its test references it.)

Preconditions (all must hold, else `None`)

- `record_type == "research_institution"` (companies excluded).
- `department_domain` not already set.
- `result["domain"]` (**institution base**) **must be present** — the hard tie to the institution domain; without it the probe bails.
- A usable `name2` (enriched, else original) that is **not** an address/location fragment and **not** a granular unit (lab/group/core/facility/program are skipped; only departments/divisions/schools/colleges/faculties are in scope).
- From `name2` it derives `tokens` (significant ≥ 3 -char words minus a generic blocklist) and an `acronym`; needs at least one.

Stages (first verified winner short-circuits)

Stage	What it does	SERP?
0 — Constructed subdomain	Builds candidate prefixes (acronym if 2–6 chars; two longest ≥ 4 -char tokens; plus their 4- and 3-char prefixes, e.g. <code>chem</code> ← <code>chemistry</code>), forms <code>{prefix}.{base}</code> , fetches all concurrently, first that verifies wins.	No (direct GET)
1 — Homepage link scrape	Fetches <code>website_url</code> (or <code>https://{base}/</code>), scores outgoing off-base links, top 5 verified.	No
2 — Site-restricted SERP	One query <code>"{cleaned} site:{base}"</code> ; score → top 5 → verify.	Yes (1)
2b — On-domain path page	Reuses stage-2 results (no new SERP). For departments hosted at a <i>path</i> (<code>clas.ufl.edu/chemistry</code>); stores the full URL .	No (reuse)
3 — Cross-domain SERP	Query <code>"{cleaned} {name1}"</code> ; skips third-party hosts; first verified host wins (e.g. <code>hopkinsmedicine.org</code> for a JHU dept).	Yes (1, if enabled)

Scoring — `_score_dept_candidate (host, base, path, title, tokens, acronym) → int`

- `needles = tokens ∪ {acronym}` (acronym only if `len ≥ 2`); no needles → 0.
- Strip base off the host; `first_seg` = the subdomain prefix.
- Generic-host prefixes (`news`, `admin`, `hr`, `library`, `careers`, `directory`, `media`, `press`, ...) → 0.
- Host prefix must match a needle** (`_seg_matches_needle`: substring, or a shared ≥ 3 -char leading prefix — `chem` ← `chemistry`), else 0. Path/title matches alone are never enough (blocks parent hosts like `fas.harvard.edu`).
- Base **3** for the host match, **+1** if a needle is in the path, **+1** if in the title. Max 5, min positive 3.

Verification gate (`_verify_candidate_url`)

Every winner must pass: fetch page, build `title + h1 + breadcrumb`, pass if the cleaned phrase (≥ 4 chars) appears verbatim, OR needle-count ≥ 2 (≥ 1 if only one needle). Rejects hosts that resolve but describe the wrong unit (e.g. `science.mit.edu` for a Computer Science query).

Output shaping

Bare-host winners (stages 0/1/2/3) are stored as `chem.ufl.edu`; stage 2b stores a full path URL. In `finalise`, **after** search terms are derived (they need the bare host), a bare host is prefixed to `https://chem.ufl.edu`.

3 · Search terms (`search_term_1`, `search_term_2`)

Computed by `derive_search_terms(result)` once in `finalise`, after all name/domain fields settle. `search_term_1` mirrors **name1 (institution)**; `search_term_2` mirrors **name2 (department)**.

search_term_1 — acronym-first, domain fallback

- 1 `_ror_acronym` (ROR's official acronym, e.g. `MIT` , `JHU`) — used verbatim.
- 2 else `strip_tld(domain)` — registrable domain minus TLD (`stanford.edu` → `stanford` , `example.co.uk` → `example` ; subdomains kept: `cs.mit.edu` → `cs.mit`).
- 3 else `derive_acronym(name1)` — a trailing parenthetical (`CS`) if present; otherwise initials of significant words (stopwords `of/for/the/and/...` skipped). Requires **every** significant token to start uppercase (else `None` — no half-acronyms) and ≥ 2 initials. → `Massachusetts Institute of Technology` → `MIT` .

Must-not-be-blank fallback (required handle): if the above is empty → the original SAP Search Term 1, else the first two significant words of `name1` (title-cased, acronyms preserved), else `None` (only when there is no name at all).

search_term_2 — unit-scoped handle

- 1 `department_domain` via `_dept_domain_to_search_term` : reduce a URL to its host, strip `www./web.` ; if it is a **subdomain of the institution base** return just the prefix (`eeecs.mit.edu` + base `mit.edu` → `eeecs`); else strip the TLD (`hopkinsmedicine.org` → `hopkinsmedicine`); a path URL `clas.ufl.edu/chemistry` → `clas` .
- 2 else **name2 text**: prefer `name2_enriched` , else `name2_original` (rejected if purely a street address). Then a trailing **parenthetical acronym** (`...(CSAIL)` → `CSAIL`), else `clean_name2_phrase` strips a generic prefix (`Department of` , `Division of` , `School of` , `Center for` , `Institute of` , `Laboratory of` , ...) and a trailing unit suffix (`Department` , `Division` , `Lab` , ...), then takes the **first two significant words**, title-cased. → `Department of Earth and Planetary Sciences` → `Earth Planetary` .
- 3 else `None` (no dept domain and no usable name2).

Accuracy caveats surfaced by the trace

1. `DEPT_PROBE_CROSS_DOMAIN` defaults to `True` despite in-code comments calling stage 3 "opt-in / off by default." So an unrestricted second SERP call normally runs when the on-domain stages fail — a cost + wrong-domain-risk lever worth confirming as intended.

2. `derive_department_domain` in `search_terms.py` is dead code — the live probe is `_probe_department_url` . If you don't want two implementations drifting, that function (and its test) could be removed.

Generated from a full source trace of `enrichment/orchestrator.py` , `enrichment/website_resolver.py` , `enrichment/tier1_ror.py` , `enrichment/tier2b_dept.py` , `enrichment/search_terms.py` , and `utils/text_utils.py` .